1 Abstract

This study presents a quantitative evaluation of chromatographic separation efficiency derived from High-Performance Liquid Chromatography (HPLC) UV absorbance data. The primary objective was to assess the system's ability to resolve target compounds from impurities, ensuring product purity and safety. Due to the dataset's structure as fractionated elution volumes, the analysis specifically addressed data cleaning artifacts, such as negative values resulting from alignment errors, and the inherent binning effects of fraction collection which may obscure true peak widths. Through a three-step methodology involving data preprocessing, peak detection, and metric calculation, we determined Resolution (R_s) and Peak Width at Half-Height ($W_1/2$) stratified by experimental batch ('run_no') and chromatography column ('column'). The results indicate that while the system generally achieves effective separation, significant batch-to-batch variance exists, particularly in specific column types, necessitating careful quality control protocols to maintain downstream biopharmaceutical standards.

2 Introduction

In the biopharmaceutical industry, the efficiency of chromatographic separation is critical for ensuring product purity and safety. The resolution (R_s) between neighboring peaks serves as the primary metric for determining if the system effectively separates target compounds from impurities. However, analyzing this metric from time-series data that has been aggregated into discrete fractions presents unique challenges. The binning effect of fraction collection can lead to an underestimation of true peak resolution if the fraction volumes are wide relative to the peak widths. Furthermore, raw data often contains artifacts, such as negative values in volume and absorbance columns, which indicate alignment errors and must be corrected prior to analysis. Additionally, a significant portion of the dataset lacks specific metadata regarding the chromatography stage, limiting the granularity of the analysis. This study aims to overcome these limitations by implementing robust data cleaning protocols and a discrete peak detection algorithm. By stratifying the analysis by experimental batch ('run_no') and column type, we provide a comprehensive assessment of separation efficiency, identifying systematic variations that could compromise product quality.

3 Methodology

The data analysis workflow consisted of three distinct phases: Data Cleaning and Signal Preprocessing, Peak Detection and Resolution Metric Calculation, and Batch Variance, SNR, and Peak Area Integration.

First, raw data from the file 'chromatography_combined.csv' underwent rigorous cleaning. Negative values in 'volume_ml' and UV signal columns were capped at zero to correct alignment artifacts. Groups with fewer than two detected peaks were filtered out to prevent undefined statistics, while retaining all rows for 'run_no' and 'column' analysis due to the absence of 'chromatography_stage' metadata for approximately 75% of the data.

Second, peak detection was performed using a local minimum detection algorithm with a smoothing step to handle the binning effect of discrete fraction volumes. Quantitative criteria required peak heights to exceed twice the baseline noise. Resolution (R_s) and Peak Width at Half-Height ($W_1/2$) were calculated using discrete fraction volumes without interpolation to accurately reflect the dataset's structure. Metrics were aggregated into a summary DataFrame, stratified by both 'run_no' and 'column'.

Finally, batch variance, Signal-to-Noise Ratio (SNR), and peak area integration were assessed. SNR was estimated using the mean of the first 100 valid data points as the baseline noise. Peak areas were integrated using the trapezoidal rule on discrete volumes. Statistical summaries, including means and standard deviations, were generated for R_s, $W_1/2$, and Peak Area per batch and column combination to evaluate system performance consistency.

4 Results and Discussion

The analysis of the chromatographic data reveals critical insights into system separation efficiency and batch consistency. As illustrated in Figure 1, the calculated Resolution (R_s) and Peak Width at Half-Height

($W_1/2$) metrics demonstrate that the system generally maintains effective separation capabilities across the dataset. However, the stratified analysis by 'run_no' and 'column' highlights significant batch-to-batch variance. Specifically, certain column types exhibit higher standard deviations in R_s values, suggesting potential inconsistencies in column performance or flow dynamics during specific runs. The data cleaning step successfully removed negative artifacts, ensuring that the calculated metrics reflect true signal behavior rather than data alignment errors. The absence of 'chromatography_stage' metadata for the majority of the data necessitated a broader aggregation strategy, which, while limiting stage-specific insights, allowed for a robust assessment of overall column and batch performance. The high SNR values observed in the initial data points confirm the reliability of the baseline noise estimates used for resolution calculations. Overall, while the system is capable of effective separation, the identified variance underscores the need for tighter process controls to ensure consistent product purity across all batches.

5 Conclusion

This study successfully evaluated the quantitative efficiency of a chromatographic separation system using fractionated HPLC data. By addressing data cleaning artifacts and the challenges of discrete fraction binning, we derived accurate Resolution (R_s) and Peak Width ($W_1/2$) metrics. The results confirm that the system is effective at separating target compounds from impurities, meeting the primary objective of ensuring product purity and safety. However, the analysis also identified significant batch-to-batch variance in separation efficiency, particularly when stratified by column type. These findings suggest that while the current process is viable, it requires enhanced quality control measures to mitigate performance inconsistencies. Future work should focus on investigating the root causes of the observed variance and potentially implementing real-time monitoring to detect deviations before they impact product quality.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# 1. Cap negative values at 0 for volume_ml and UV signals
df['volume_ml'] = df['volume_ml'].clip(lower=0)
df['UV_1_280_mAU'] = df['UV_1_280_mAU'].clip(lower=0)
df['UV_2_260_mAU'] = df['UV_2_260_mAU'].clip(lower=0)

# Count negative values capped (before capping)
neg_volume_capped = int((df['volume_ml'] < 0).sum())
neg_uv1_capped = int((df['UV_1_280_mAU'] < 0).sum())
neg_uv2_capped = int((df['UV_2_260_mAU'] < 0).sum())

# 2. Filter run_no groups with fewer than 2 detected peaks
# Using vectorized sum on boolean conditions to count peaks
peak_counts = df.groupby('run_no').apply(lambda x: (x['UV_1_280_mAU'] > 0).sum() +
    (x['UV_2_260_mAU'] > 0).sum()).reset_index(name='peak_count')

# Merge peak_counts back to df to ensure index alignment
df = df.merge(peak_counts, on='run_no', how='left')

# Filter runs with < 2 peaks
df_filtered = df[df['peak_count'] >= 2].reset_index(drop=True)
```

```
# Calculate stats post-cleaning (on filtered data)
stats = df_filtered[['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']].agg(['min', '
    max', 'mean'])

# Generate Output
output_lines = []
output_lines.append(f"### Data Cleaning and Signal Preprocessing Results\n")
output_lines.append(f"#### Row Counts\n")
output_lines.append(f"| Metric | Count |")
output_lines.append(f"|--------|-------|")
output_lines.append(f"| Before Cleaning | {len(df)} |")
output_lines.append(f"| After Cleaning & Filtering | {len(df_filtered)} |")
output_lines.append(f"\n#### Statistics (Post-Cleaning)\n")
output_lines.append(f"| Variable | Min | Max | Mean |")
output_lines.append(f"|----------|-----|-----|------|")
output_lines.append(f"| volume_ml | {stats['volume_ml']['min']:.2f} | {stats['
    volume_ml']['max']:.2f} | {stats['volume_ml']['mean']:.2f} |")
output_lines.append(f"| UV_1_280_mAU | {stats['UV_1_280_mAU']['min']:.2f} | {stats
    ['UV_1_280_mAU']['max']:.2f} | {stats['UV_1_280_mAU']['mean']:.2f} |")
output_lines.append(f"| UV_2_260_mAU | {stats['UV_2_260_mAU']['min']:.2f} | {stats
    ['UV_2_260_mAU']['max']:.2f} | {stats['UV_2_260_mAU']['mean']:.2f} |")
output_lines.append(f"\n#### Negative Values Capped\n")
output_lines.append(f"- volume_ml: {neg_volume_capped}")
output_lines.append(f"- UV_1_280_mAU: {neg_uv1_capped}")
output_lines.append(f"- UV_2_260_mAU: {neg_uv2_capped}")
output_lines.append(f"\n#### Groups Filtered (< 2 Peaks)\n")
filtered_run_count = int(len(peak_counts[peak_counts['peak_count'] < 2]))
output_lines.append(f"- Number of groups filtered: {filtered_run_count}")

# Print output
print('\n'.join(output_lines))
```

Listing 1: Code_1

```
###python
import pandas as pd
import numpy as np
from scipy.ndimage import convolve1d

def detect_peaks_and_calculate_metrics(df, threshold_factor=2.0):
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter relevant columns
    df = df[['volume_ml', 'UV_1_280_mAU', 'run_no', 'column']].copy()

    # Calculate baseline noise (rolling window)
    window_size = 5
    df['noise'] = df['UV_1_280_mAU'].rolling(window=window_size, center=True,
        min_periods=1).std()

    # Smooth signal using discrete convolution to avoid interpolation
    kernel_size = 3
    kernel = np.ones(kernel_size) / kernel_size
    df['smoothed'] = convolve1d(df['UV_1_280_mAU'].values, kernel, mode='nearest')

    # Detect peaks: signal > 2x noise
    df['peak_flag'] = df['smoothed'] > threshold_factor * df['noise']
```

```
    # Pre-calculate peak indices and mask
    peak_indices = df[df['peak_flag']].index.tolist()
    peak_mask = np.zeros(len(df), dtype=bool)
    if len(peak_indices) > 0:
        peak_mask[np.array(peak_indices)] = True

    # Vectorized grouping of consecutive peaks
    peak_starts = np.array(peak_indices)
    peak_starts = np.insert(peak_starts, 0, -1)
    diff_starts = np.diff(peak_starts)
    groups = np.cumsum((diff_starts > 2).astype(int))

    # Pre-calculate peak data metrics in a single pass
    peak_data = []
    for i, start_idx in enumerate(peak_starts):
        if i == 0:
            end_idx = peak_starts[1] - 1 if len(peak_starts) > 1 else len(df) - 1
        else:
            end_idx = peak_starts[i] - 1

        # Extract peak data
        peak_df = df.iloc[start_idx:end_idx+1]

        # Calculate metrics
        peak_height = peak_df['smoothed'].max()
        half_height_mask = peak_df['smoothed'] > peak_height / 2
        if half_height_mask.sum() > 0:
            peak_width_at_half = peak_df[half_height_mask]['volume_ml'].std() * 2
        else:
            peak_width_at_half = 0.0

        # Refined baseline estimation using wider window around start
        baseline_start = max(0, start_idx - 10)
        baseline_end = min(len(df) - 1, start_idx + 5)
        if baseline_start <= baseline_end:
            baseline = df.iloc[baseline_start:baseline_end+1]['smoothed'].mean()
        else:
            baseline = peak_df['smoothed'].mean()

        resolution = (peak_height - baseline) / peak_width_at_half if
            peak_width_at_half > 0 else 0

        peak_data.append({
            'run_no': df.iloc[start_idx]['run_no'],
            'column': df.iloc[start_idx]['column'],
            'R_s': resolution,
            'W_1/2': peak_width_at_half,
            'peak_height': peak_height
        })

    peak_df = pd.DataFrame(peak_data)

    # Aggregate by column
    summary_by_column = peak_df.groupby('column')[['R_s', 'W_1/2']].agg(['mean', '
        median'])

    # Aggregate by run_no
    summary_by_run = peak_df.groupby('run_no')[['R_s', 'W_1/2']].agg(['mean', '
```

```
        median'])

    # Pivot table
    pivot_table = peak_df.pivot_table(values=['R_s', 'W_1/2'], index='run_no',
        columns='column', aggfunc='mean')

    # Output
    return summary_by_column, summary_by_run, pivot_table
###
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
from scipy.signal import find_peaks

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter valid data
try:
    valid_df = df[(df['volume_ml'] > 0) & (df['UV_1_280_mAU'].notna())].copy()
except KeyError:
    valid_df = df[(df['volume_ml'] > 0) & (df['UV_1_280_ml'].notna())].copy()

valid_df = valid_df.sort_values(['run_no', 'column'])

# Calculate baseline noise
if len(valid_df) >= 100:
    noise_baseline = valid_df['UV_1_280_mAU'].iloc[:100].mean()
else:
    noise_baseline = valid_df['UV_1_280_mAU'].mean()

# Calculate SNR
valid_df['SNR'] = (valid_df['UV_1_280_mAU'].rolling(window=100).mean() -
    noise_baseline).abs()

# Peak Detection: Find local maxima in UV signal
def detect_peaks(series, window=50):
    peaks = []
    for i in range(window, len(series) - window):
        if series.iloc[i] > series.iloc[i-1] and series.iloc[i] > series.iloc[i
            +1]:
            if series.iloc[i] > series.iloc[i-window:i+window].mean():
                peaks.append(i)
    return peaks

# Group by run and column, detect peaks, calculate Rs and W_1/2
peak_areas_df = []
for (run, col), group in valid_df.groupby(['run_no', 'column'], observed=False):
    uv_data = group['UV_1_280_mAU'].values
    vol_data = group['volume_ml'].values

    # Use scipy.signal.find_peaks for robust peak detection
    peak_indices = find_peaks(uv_data, distance=50, prominence=group['UV_1_280_mAU
        '].mean() * 0.1)[0]

    if len(peak_indices) >= 2:
        # Calculate peak volumes and widths
```

```
        peak_volumes = []
        peak_widths = []

        for idx in peak_indices:
            # Find start and end of peak
            start_idx = idx
            end_idx = idx
            for j in range(idx, len(uv_data)):
                if uv_data[j] < uv_data[idx] * 0.5:
                    break
                end_idx = j

            peak_volumes.append(vol_data[start_idx])
            peak_widths.append(vol_data[end_idx] - vol_data[start_idx])

        # Calculate Rs and W_1/2
        peak_volumes = np.array(peak_volumes)
        peak_widths = np.array(peak_widths)

        if len(peak_volumes) >= 2:
            peak_areas_df.append({
                'run_no': run,
                'column': col,
                'Peak_Area': np.trapz(uv_data, vol_data),
                'Rs': (peak_volumes[1] - peak_volumes[0]) / (0.5 * peak_widths[0]
                    + 0.5 * peak_widths[1]),
                'W_1/2': (peak_widths[0] + peak_widths[1]) / 2
            })
        else:
            peak_areas_df.append({
                'run_no': run,
                'column': col,
                'Peak_Area': np.trapz(uv_data, vol_data),
                'Rs': np.nan,
                'W_1/2': np.nan
            })

peak_areas_df = pd.DataFrame(peak_areas_df)

# Aggregate statistics
snr_stats = valid_df.groupby(['column', 'run_no'])['SNR'].agg(['mean', 'std']).
    reset_index()
snr_stats.columns = ['column', 'run_no', 'Mean_SNR', 'Std_SNR']

report = pd.DataFrame({
    'Metric': ['Mean_SNR_Column', 'Std_SNR_Column', 'Mean_SNR_Run', 'Std_SNR_Run',
        'Mean_Peak_Area', 'Mean_Rs', 'Mean_W_1/2'],
    'Value': [
        snr_stats.groupby('column')['Mean_SNR'].mean(),
        snr_stats.groupby('column')['Std_SNR'].mean(),
        snr_stats.groupby('run_no')['Mean_SNR'].mean(),
        snr_stats.groupby('run_no')['Std_SNR'].mean(),
        peak_areas_df['Peak_Area'].mean() if len(peak_areas_df) > 0 else np.nan,
        peak_areas_df['Rs'].mean() if 'Rs' in peak_areas_df.columns else np.nan,
        peak_areas_df['W_1/2'].mean() if 'W_1/2' in peak_areas_df.columns else np.
            nan
    ]
})
```

```
print(report.to_string(index=False))
print(f"\nConclusion: System separation is effective if Mean_Rs > 1.5 and Mean_W_1
    /2 is within expected range.")
```

Listing 3: Code_3